No Message Left Behind

What if your messages are delivered reliably without any fancy, complex tools and maintenance effort?

Stefan Tomm

- Professional Software Engineer by heart since 2008
- at meshcloud since 2017 building a Sovereign Internal Developer Platform
- Actively developing and improving our product every day with the team
- Currently fascinated by how to improve our Dev process with AI efficiently

Why this talk?

- We needed a **simple but reliable** messaging solution at meshcloud
- Pieces existed in blog posts — wiring them together end-to-end took quite some effort
- A **complete, working example** — not just theory and the happy path

Setting the Scene

center

> **But what can go wrong?** *Spoiler: quite a lot.*

The Happy Path

Failure on Producer Side

DB and broker fall out of sync

center

The Naive Approach

Publish the event **inside** the open DB transaction.

center

> Looks safe — message is only published after the order is already saved. What could go wrong?

The Transaction Can Still Fail

> Order not saved, but Shipping will **process** the **event anyway**

Publish After the Transaction?

Move the publish **outside** the transaction to eliminate the rollback risk.

center

> Order saved in DB, but shipping **never notified**
> **Same silent data loss** — just a different failure window

What About Two-Phase Commit (2PC)?

Coordinate DB and broker atomically via a distributed transaction protocol.

Complexity	Requires an XA transaction manager across two systems
Performance	Blocking protocol — degrades throughput significantly
RabbitMQ	No XA / 2PC support out of the box

Too much complexity, too little payoff — **there is a simpler way.**

Solution: The Transactional Outbox

> Write the event to an **outbox table in the same DB transaction**

center

Outbox: What about concurrency?

`SELECT … FOR UPDATE SKIP LOCKED` ensures:

- Each instance locks only the rows it claims
- Other instances **skip** locked rows — no waiting, no double-publishing

center

Scales horizontally with **zero coordination overhead**.

Outbox: What about failures?

center

> The outbox makes events **durable** — survives crashes, restarts, and broker outages.

✅ Producer Problem — Solved

Failure on Consumer Side

The consumer can't process the message

center

Naive Thought

"RabbitMq will handle it for us automatically."

No, it can either **requeue** directly or write to a **Dead Letter Exchange**.

Naive fix: Requeue immediately

center

Problems:

- Burns CPU & network for nothing
- Potentially overwhelms an already struggling dependency
- Pollutes logs with thousands of identical errors

> We need to **back off** between retries.

What we really want: Exponential backoff

center

RabbitMQ has no native delayed delivery.

→ Build it from standard features.

Dead-Letter Exchange (DLX) & Dead Letter Queue (DLQ)

A DLX can be defined on a queue for rejected & expired messages.

center

Message TTL

A queue can be configured with a **TTL** (time-to-live) for its messages.

center

> **TTL + DLX = delay line** - Standard RabbitMQ, no plugins needed.

Solution: The Retry Ladder

Why separate queues per TTL level?

> **TTL expiry is only checked at the head of the queue (FIFO).**

→ **One dedicated queue per TTL level.**

How is the routing to retry queues done?

An **interceptor** wraps around the original message listener:

center

A note on acknowledgment

center

Reminder: The Retry Ladder

The `retry-wait-ended` queue

It simply routes back to the original queue (see `RetryWaitEndedRabbitListener`)

```
@RabbitListener(queues = ["retry-wait-ended"])
fun onMessage(message: Message) {
    val exchange   = headers["x-original-exchange"]    as String
    val routingKey = headers["x-original-routing-key"] as String

    rabbitTemplate.send(exchange, routingKey, message) // back to original queue
}
```

Optional: Two layers of retry

See `ShippingRabbitConfig` in code

✅ Consumer Problem — Solved

center

The Complete Picture

Production Reality

🙂 It's boring (in the best way)

No special infrastructure · Standard RabbitMQ only · Minimal maintenance

📊 What to monitor

- **Outbox table** growing? → broker issue
- **retry-3/4/5** filling up? → real outage
- **retry-5 depth** > 0? → human intervention needed

⚠️ What it's not

- Not exactly-once
- Not zero-latency *(outbox ~1s; retries add minutes)*

Demo Time! 🚀

Let's see it in action

sheep center

Thank You!

QR code
GitHub Repo

Full source code & presentation on GitHub:

https://github.com/meshcloud/resilient-rabbitmq-demo

Further reading

- RabbitMQ Dead-Letter Exchanges — rabbitmq.com/docs/dlx
- Transactional Outbox Pattern — microservices.io/patterns/data/transactional-outbox